

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..38

CONFIGURATION

 Copy as Markdown

Configuration

You configure future calls in your Serverpod config files or through environment variables. The options and their environment-variable names are listed in full in the [Configuration reference](#).

Option	Default	Controls
<code>futureCallExecutionEnabled</code>	<code>true</code>	Whether this server runs future calls at all.
<code>futureCall.concurrencyLimit</code>	<code>1</code>	How many calls may run at once.
<code>futureCall.scanInterval</code>	<code>5000</code>	How often, in milliseconds, the server checks for due calls.
<code>futureCall.checkBrokenCalls</code>	<code>unset</code>	Whether to scan for broken calls on startup.
<code>futureCall.deleteBrokenCalls</code>	<code>false</code>	Whether to delete broken calls that are found.

```
futureCallExecutionEnabled: true
```

```
futureCall:
```

```
  concurrencyLimit: 1      # default
  scanInterval: 5000      # default, in milliseconds
```



Execution options

Enable or disable execution

The `futureCallExecutionEnabled` option turns future call execution on or off for a server. It is `true` by default. Set it to `false` in environments where background tasks should not run, such as a staging server where you want to test API behavior without triggering scheduled work.

```
futureCallExecutionEnabled: false
```

Concurrency limit

The `concurrencyLimit` option sets how many future calls may run at the same time. The default is `1`, meaning calls run one after another. Raise it to run more calls in parallel, or keep it small so future calls do not crowd out other work on a busy server.

Set it to `0` or a negative number to remove the limit entirely, allowing unlimited concurrent calls. Use this with care, since a burst of due calls can then exhaust the server's resources.

```
futureCall:  
  concurrencyLimit: 5
```

Scan interval

The `scanInterval` option sets how often, in milliseconds, the server checks the database for calls that are due. The default is `5000` (5 seconds). A shorter interval runs calls closer to their scheduled time. A longer one reduces database load.

```
futureCall:  
  scanInterval: 2000
```

Broken future calls

A scheduled call becomes broken when the code needed to run it no longer matches what was stored. This happens if, before the call runs, you restart the server after:

- Removing the method the call points to.
- Changing a future call method's signature so the stored parameters no longer deserialize.
- Changing a model used as a parameter so the stored data no longer deserializes.

A broken call fails at runtime when the server tries to execute it. You are responsible for keeping calls backward compatible, but Serverpod can detect and remove broken calls for you.

Check for broken calls

On startup the server can scan for broken calls. Whether it does depends on `checkBrokenCalls`:

- **Unset (the default):** the check runs only when the table holds fewer than 1000 future calls, so a large backlog does not slow startup.
- `true`: the check always runs, regardless of how many calls are scheduled.
- `false`: the check never runs.

```
futureCall:  
  checkBrokenCalls: true
```

NOTE

The `maintenance` role also runs this check. Started in that role, the server checks for broken calls, runs every currently-due future call once, and then exits. This is how future calls get processed in environments that do not run a persistent server, such as serverless hosting. See [server roles](#).

```
dart run bin/main.dart --role maintenance
```

The process exits normally whether or not broken calls are found, so read the result from the logs.

WARNING

This role executes every due future call, so it is not a dry check. Do not run it against a production database to see what would happen.

Delete broken calls

When the check finds broken calls it logs a warning but does not delete them. Set `deleteBrokenCalls` to `true` to delete them instead. Deletion only happens when the check actually runs.

```
futureCall:  
  deleteBrokenCalls: true
```

Related

- [Future calls](#): defining and scheduling a call.
- [Configuration reference](#): every key with its environment variable.
- [Server roles](#): the maintenance role used above.

